

Лекция 1

Булевы функции и их формы записи

Опр Булева функция — функция вида: $\{0,1\}^n \rightarrow \{0,1\}$

Пусть $f(x_1, x_2, \dots, x_n)$ — булева функция от n переменных. Тогда всего 2^n наборов.

Булевы функции задаются таблицами истинности.

Опр Переменная x_i называется существенной для функции $f(x_1, x_2, \dots, x_n)$; $i \in \overline{1, n}$, если $\exists$ набор значений $d_1, d_2, \dots, d_{i-1}, d_{i+1}, \dots, d_n \in \{0,1\}$ такой, что $f(d_1, d_2, \dots, d_{i-1}, 0, d_{i+1}, \dots, d_n) \neq f(d_1, d_2, \dots, d_{i-1}, 1, d_{i+1}, \dots, d_n)$.

В противном случае переменная называется фиктивной (несущественной).

Булевы функции от двух переменных (x, y): 0, 1 (тождественные ложь и истина), $x, y, \bar{x}, \bar{y}$ (инверсия/отрицание), $x \vee y$ (дизъюнкция), $x \wedge y$ (конъюнкция), $x \rightarrow y$ (импликация), $x \equiv y$ (эквивалентность/равенство), $x \oplus y$ (XOR), $x \uparrow y$ (штрих Шеффера), $x \downarrow y$ (стрелка Пирса)

Таблица истинности для данных функций:

$x \downarrow y$	$x \uparrow y$	$x \oplus y$	$x \equiv y$	$x \leftarrow y$	$x \wedge y$	$x \vee y$	$\bar{y}$	$\bar{x}$	y	x	0	1
1	1	0	1	1	0	0	1	1	0	0	0	0
0	1	1	0	1	0	1	0	1	0	0	1	0
0	1	1	0	0	1	1	0	0	1	0	1	0
0	1	1	0	0	1	1	0	0	1	0	1	0
0	0	0	1	1	1	1	0	0	1	1	0	1
0	0	0	1	1	1	1	0	0	1	1	0	1
0	0	0	1	1	1	1	0	0	1	1	0	1
0	0	0	1	1	1	1	0	0	1	1	0	1
0	0	0	1	1	1	1	0	0	1	1	0	1
0	0	0	1	1	1	1	0	0	1	1	0	1
0	0	0	1	1	1	1	0	0	1	1	0	1
0	0	0	1	1	1	1	0	0	1	1	0	1
0	0	0	1	1	1	1	0	0	1	1	0	1
0	0	0	1	1	1	1	0	0	1	1	0	1
0	0	0	1	1	1	1	0	0	1	1	0	1
0	0	0	1	1	1	1	0	0	1	1	0	1
0	0	0	1	1	1	1	0	0	1	1	0	1
0	0	0	1	1	1	1	0	0	1	1	0	1
0	0	0	1	1	1	1	0	0	1	1	0	1
0	0	0	1	1	1	1	0	0	1	1	0	1
0	0	0	1	1	1	1	0	0	1	1	0	1
0	0	0	1	1	1	1	0	0	1	1	0	1
0	0	0	1	1	1	1	0	0	1	1	0	1
0	0	0	1	1	1	1	0	0	1	1	0	1
0	0	0	1	1	1	1	0	0	1	1	0	1
0	0	0	1	1	1	1	0	0	1	1	0	1
0	0	0	1	1	1	1	0	0	1	1	0	1
0	0	0	1	1	1	1	0	0	1	1	0	1
0	0	0	1	1	1	1	0	0	1	1	0	1
0	0	0	1	1	1	1	0	0	1	1	0	1
0	0	0	1	1	1	1	0	0	1	1	0	1
0	0	0	1	1	1	1	0	0	1	1	0	1
0	0	0	1	1	1	1	0	0	1	1	0	1
0	0	0	1	1	1	1	0	0	1	1	0	1
0	0	0	1	1	1	1	0	0	1	1	0	1
0	0	0	1	1	1	1	0	0	1	1	0	1
0	0	0	1	1	1	1	0	0	1	1	0	1
0	0	0	1	1	1	1	0	0	1	1	0	1
0	0	0	1	1	1	1	0	0	1	1	0	1
0	0	0	1	1	1	1	0	0	1	1	0	1
0	0	0	1	1	1	1	0	0	1	1	0	1
0	0	0	1	1	1	1	0	0	1	1	0	1
0	0	0	1	1	1	1	0	0	1	1	0	1
0	0	0	1	1	1	1	0	0	1	1	0	1
0	0	0	1	1	1	1	0	0	1	1	0	1
0	0	0	1	1	1	1	0	0	1	1	0	1
0	0	0	1	1	1	1	0	0	1	1	0	1
0	0	0	1	1	1	1	0	0	1	1	0	1
0	0	0	1	1	1	1	0	0	1	1	0	1
0	0	0	1	1	1	1	0	0	1	1	0	1
0	0	0	1	1	1	1	0	0	1	1	0	1
0	0	0	1	1	1	1	0	0	1	1	0	1
0	0	0	1	1	1	1	0	0	1	1	0	1
0	0	0	1	1	1	1	0	0	1	1	0	1
0	0	0	1	1	1	1	0	0	1	1	0	1
0	0	0	1	1	1	1	0	0	1	1	0	1
0	0	0	1	1	1	1	0	0	1	1	0	1
0	0	0	1	1	1	1	0	0	1	1	0	1
0	0	0	1	1	1	1	0	0	1	1	0	1
0	0	0	1	1	1	1	0	0	1	1	0	1
0	0	0	1	1	1	1	0	0	1	1	0	1
0	0	0	1	1	1	1	0	0	1	1	0	1
0	0	0	1	1	1	1	0	0	1	1	0	1
0	0	0	1	1	1	1	0	0	1	1	0	1
0	0	0	1	1	1	1	0	0	1	1	0	1
0	0	0	1	1	1	1	0	0	1	1	0	1
0	0	0	1	1	1	1	0	0	1	1	0	1
0	0	0	1	1	1	1	0	0	1	1	0	1
0	0	0	1	1	1	1	0	0	1	1	0	1
0	0	0	1	1	1	1	0	0	1	1	0	1
0	0	0	1	1	1	1	0	0	1	1	0	1
0	0	0	1	1	1	1	0	0	1	1	0	1
0	0	0	1	1	1	1	0	0	1	1	0	1
0	0	0	1	1	1	1	0	0	1	1	0	1
0	0	0	1	1	1	1	0	0	1	1	0	1
0	0	0	1	1	1	1	0	0	1	1	0	1
0	0	0	1	1	1	1	0	0	1	1	0	1
0	0	0	1	1	1	1	0	0	1	1	0	1
0	0	0	1	1	1	1	0	0	1	1	0	1
0	0	0	1	1	1	1	0	0	1	1	0	1
0	0	0	1	1	1	1	0	0	1	1	0	1
0	0	0	1	1	1	1	0	0	1	1	0	1
0	0	0	1	1	1	1	0	0	1	1	0	1
0	0	0	1	1	1	1	0	0	1	1	0	1
0	0	0	1	1	1	1	0	0	1	1	0	1
0	0	0	1	1	1	1	0	0	1	1	0	1
0	0	0	1									

Опр Булевы формулы:

- 1) $\forall$ переменная, 0, 1 - формулы
- 2) Если A - формула, то $\bar{A}$ - формула
- 3) Если A и B формулы, то $(A \vee B); (A \wedge B); \dots (A \downarrow B)$ - формулы
- 4) Других формул нет.

Опр КНФ - конъюнктивная нормальная форма, или конъюнкция дизъюнкций.

Пример: $(x_1 \vee \bar{x}_2 \vee x_3) \wedge (\bar{x}_2 \vee x_1 \vee x_2) \wedge (x_1 \vee x_4)$

Опр Литерал - переменная или её отрицание.

Опр КНФ такая, что каждый дизъюнкт содержит все переменные ровно по одному разу, и все дизъюнкты различны, называется совершенной конъюнктивной нормальной формой (СКНФ)

СКНФ: $\bigwedge (x_1^{b_1} \vee x_2^{b_2} \vee \dots \vee x_n^{b_n})$, где $x_i^{b_i} = \begin{cases} x_i, & b_i = 1 \\ \bar{x}_i, & b_i = 0 \end{cases}$

СКНФ $\exists$ для $\forall$ функции кроме тождественной 1 и строится по булевым строкам

Опр ДНФ - дизъюнктивная нормальная форма, или дизъюнкция конъюнкций

Опр ДНФ такая, что каждый конъюнкт содержит все переменные ровно по одному разу, и все конъюнкты различны называется

совершенной дизъюнктивной нормальной формой (СДНФ)

$$\text{СДНФ: } \bigvee (x_1^{b_1} \wedge x_2^{b_2} \wedge \dots \wedge x_n^{b_n})$$

СДНФ $\exists$ для $\forall$ функции кроме тождественного 0 и строится по единичным строкам.

Т.к. $\forall$ функция представлена в виде СКНФ или СДНФ, то $\forall$ функцию можно выразить, используя лишь инверсии, конъюнкции и дизъюнкции.

Пример СКНФ и СДНФ по таблице истинности:

x	y	z	f
0	0	0	1
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	0

$$\text{СКНФ: } (x \vee y \vee \bar{z}) \wedge (x \vee \bar{y} \vee z) \wedge (\bar{x} \vee y \vee z) \wedge (\bar{x} \vee y \vee \bar{z}) \wedge (\bar{x} \vee \bar{y} \vee \bar{z})$$

$$\text{СДНФ: } (\bar{x} \wedge \bar{y} \wedge \bar{z}) \vee (\bar{x} \wedge y \wedge z) \vee (x \wedge y \wedge \bar{z})$$

Если $\exists$ СКНФ (СДНФ) для булевой функции, то она единственна.

□ Следует из построения ■

Допустимые действия с формулами:

1) Замена переменных без отождествления

$$x_i \rightarrow y_i$$

2) Замена переменных с отождествлением

$$x, y \rightarrow z \text{ или } x, y \rightarrow y \text{ (можно две переменные обозначить одной)}$$

3) Суперпозиция функций (композиция, подстановка)

$$f_1(x; y); f_2(a; b; c)$$

$$\Rightarrow f_2(f_1(x; y); b; c)$$

Зам. $a = f_1(x; y)$

Опр Набор функций называется полным, если $\forall$ функция выражена через них при помощи суперпозиции и замены переменных в том числе с отождествлениями.

Примеры полных наборов: $\{\bar{x}; x \vee y\}$; $\{1, x \oplus y; x \wedge y\}$

Опр Форма функции вида $a_0 \oplus a_1 x_1 \oplus a_2 x_2 \oplus \dots \oplus a_{2^n-1} x_1 x_2 \dots x_n$; $a_i \in \{0, 1\}$ называется полиномом Жегалкина.

Для $\forall$ функции $\exists$ полином Жегалкина

Утв Полином Жегалкина единственный

□ $f(x_1; x_2 \dots x_n); p(x_1; x_2 \dots x_n); q(x_1; x_2 \dots x_n)$ - различные функции, ирим $p \oplus q = f \oplus f = 0$.

Найдём у p и q младший несовпадающий элемент $x_{i_1} x_{i_2} \dots x_{i_k}$

Тогда в $p \oplus q$ перед ним 1, т.е. $x_j = 1 \Leftrightarrow j \in i_1 i_2 \dots i_k$ (*)

Но тогда $p \oplus q \neq 0$. Противоречие, значит $p = q$. ■

Здесь p и q по предположению разные полиномы Жегалкина для функции f

(*) Подставим в функцию набор $x_{i_1} = x_{i_2} = \dots = x_{i_k} = 1$, остальные переменные $= 0$

Тогда на этом наборе значения p и q не совпадают.

Лекция 2

Классы и критерий Поста

Опр Класс функций K замкнут, если $\forall$ суперпозиция функций из данного класса и замена переменных в том числе с отождествлением даёт функцию из этого же класса.

Опр Классы Поста - следующие замкнутые классы функций:

- 1) $T_0 = \{f \mid f(0, 0, \dots, 0) = 0\}$
- 2) $T_1 = \{f \mid f(1, 1, \dots, 1) = 1\}$
- 3) $L = \{f \mid a_0 \oplus a_1 x_1 \oplus \dots \oplus a_n x_n\}$ - линейные полиномы Ж. Галкина
- 4) $S = \{f \mid f(\bar{x}_1, \bar{x}_2, \dots, \bar{x}_n) = \bar{f}(x_1, x_2, \dots, x_n)\}$ - самодвойственные функции
- 5) $M = \{f \mid \forall \alpha, \beta \in \{0, 1\}^n : \alpha \geq \beta \Rightarrow f(\alpha) \geq f(\beta)\}$ - монотонные функции

$\alpha \geq \beta$, если $\forall i \in \overline{1, n} : \alpha_i \geq \beta_i$

Теорема (Критерий Поста)

K -полный набор функций $\Leftrightarrow K$ не содержится целиком ни в одном из классов Поста ($T_0; T_1; L; S; M$)

$\square \Rightarrow$ Пусть K лежит в одном из классов Поста, б.о.о. в T_0 . Т.к.

T_0 -замкнут, то $\forall$ функция, получаемая суперпозицией и заменой переменных с возможным отождествлением лежит в $K \Rightarrow$ лежит в T_0 .

Т.к. T_0 не совпадает со множеством всех функций алгебры логики $\Rightarrow$

не все функции могут быть получены из $K \Rightarrow K$ -не полный. Противоречие.

⊆ Т.к. K не лежит ни в одном из классов Поста, то $\exists$ функции: $f_{\bar{0}}, f_{\bar{1}}, f_{\bar{2}}, f_{\bar{M}}, f_{\bar{S}} \in K$: $f_{\bar{0}} \notin T_0$; $f_{\bar{1}} \notin T_1$; $f_{\bar{2}} \notin L$; $f_{\bar{M}} \notin M$; $f_{\bar{S}} \notin S$.

P -м функцию $f_{\bar{0}}(x_1; x_2 \dots x_n)$. Сделаем замену с отождествлением:
 $f_{\bar{0}}(x; x \dots x) = \varphi_{\bar{0}}(x)$. При этом $\varphi_{\bar{0}}(0) = 1$; $\varphi_{\bar{0}}(1) = ? \Rightarrow$
 $\Rightarrow$ это функция $\varphi_{\bar{0}}(x) = 1$, если $\varphi_{\bar{0}}(1) = 1$ или $\varphi_{\bar{0}}(x) = \bar{x}$, если $\varphi_{\bar{0}}(1) = 0$.

P -м функцию $f_{\bar{1}}(x_1; x_2 \dots x_n)$. Сделаем замену с отождествлением:
 $f_{\bar{1}}(x; x \dots x) = \varphi_{\bar{1}}(x)$. При этом $\varphi_{\bar{1}}(1) = 0$; $\varphi_{\bar{1}}(0) = ? \Rightarrow$
 это функция $\varphi_{\bar{1}}(x) = 0$, если $\varphi_{\bar{1}}(0) = 0$ или $\varphi_{\bar{1}}(1) = \bar{x}$, если $\varphi_{\bar{1}}(0) = 1$.

Мы хотим получить набор из всех трёх функций: $0; 1; \bar{x}$.

P -м все 4 варианта возможных наборов функций, полученных из T_0 и T_1 :

- 1) Получено 1 и $\bar{x}$. Тогда получаем из них 0 , т.к. $\bar{1} = 0$.
- 2) Получено $\bar{x}$ и 0 . Тогда получаем из них 1 , т.к. $\bar{0} = 1$.
- 3) Получено $\bar{x}$.

P -м $f_{\bar{S}} \notin S$. Тогда $\exists d \in \{0, 1\}^n$: $f_{\bar{S}}(d) = f_{\bar{S}}(I)$, если d и I сравнимы.

Пусть $f_{\bar{S}}(x^{d_1}; x^{d_2} \dots x^{d_n}) = f_{\bar{S}}(x; x \dots x) = \varphi_{\bar{S}}(x)$ (замена с отождествлением)

Тогда $\varphi_{\bar{S}}(0) = f_{\bar{S}}(d)$

$\varphi_{\bar{S}}(1) = f_{\bar{S}}(I) \Rightarrow \varphi_{\bar{S}}(0) = \varphi_{\bar{S}}(1) \Rightarrow \varphi_{\bar{S}}(x) = \text{const.}$

Аналогично п. 1 и 2 из $\bar{x}$ и const получаем группу const , т.е. получим весь набор $0, 1, \bar{x}$

4) Получено 1 и 0.

Р-м $f_{\bar{a}} \notin M. \Rightarrow \exists \alpha \geq \beta: f_{\bar{a}}(\alpha) < f_{\bar{a}}(\beta)$, т.е. $f_{\bar{a}}(\alpha) = 0, f_{\bar{a}}(\beta) = 1$

Находим путь по диаграмме Харриса от α до β . На каком-то шаге будет ребро $a \rightarrow b$, отличающееся одним разрядом такое, что $f_{\bar{a}}(a) < f_{\bar{a}}(b)$ и $a \geq b$

Пусть этот различающийся разряд - t . Тогда $a_t = 1, b_t = 0$. Тогда Можно, т.к. есть 0 и 1
 $f_{\bar{a}}$ Подставим в $f_{\bar{a}}$ следующее: $\begin{cases} x_i = 1; a_i = 1 \\ x_i = 0; a_i = 0 \end{cases} \forall i \neq t$. А для t

подставляем 0 и 1. Получаем $\varphi_{\bar{a}}(x)$; $\varphi_{\bar{a}}(0) \neq \varphi_{\bar{a}}(1)$, но $\varphi_{\bar{a}}(1) = f_{\bar{a}}(a) = 0$, $\varphi_{\bar{a}}(0) = f_{\bar{a}}(b) = 1 \Rightarrow$ получим $\bar{x}$

Таким образом, используя классы T_0, T_i, S и M получаем функции $0, 1$ и $\bar{x}$.

Теперь р-м $f_{\bar{z}} \in L$. $f_{\bar{z}} = d_0 \oplus d_1 x_1 \oplus \dots \oplus d_{2^k-1} x_1 x_2 \dots x_k$. Т.к. $f_{\bar{z}} \in L$, то

$\exists \min t: d_t = 1$ и слагаемое с ним $x_{i_1} x_{i_2} \dots x_{i_k}$ содержит $k-2$ множителей. ос. перем = 0

Подставим $x_{i_1} = x, x_{i_2} = y, x_{i_3} = x_{i_4} = \dots = x_{i_k} = 1$. Разрешено. Р-м

все варианты такой функции после подстановки $\odot$ см. сл. стр.

$xy + x + y + 1$
 $xy + x + y$
 $xy + x + 1$
 $xy + x$
 $xy + y + 1$
 $xy + y$
 $xy + 1$
 xy

~~Если есть слагаемое xy , то прибавим полученные ранее функции $0, 1$ или $\bar{x}, \bar{y}$, если надо, чтобы избавиться от слагаемых. Мы получили конъюнкцию.~~

Набор $\bar{x}$ и xy - полный k -битовый набор $\blacksquare$

* Продолжение доказательства критерия Поста.

Мы получили одну из набора функций. Получим из него конъюнкцию или дизъюнкцию. Для начала научимся избавляться от 1, если она есть. Заметим, что если взять отрицание от функции, то 1 исчезнет. Р-м оставшиеся случаи:

1) $x \vee y$ - готово

2) $x \vee x \vee y = x \vee y$

3) $x \vee x = x(y \vee \bar{y}) = x \bar{y} \rightarrow$ замена $z = \bar{y}$.

$x \vee y$ аналогично.

Таким образом, мы получили отрицание и дизъюнкцию или конъюнкцию, т.е. полный набор $\Rightarrow K$ полный набор ■

Лекция 3

Семантическое и синтаксическое следствие. Формальные системы.

Исчисление высказываний.

Сравнение функций на языке алгебры логики и теории множеств:

$$A \subseteq U : a(x) = \begin{cases} 1; x \in A \\ 0; x \notin A \end{cases} ; \quad B \subseteq U : b(x) = \begin{cases} 1; x \in B \\ 0; x \notin B \end{cases}$$

U - множество всех рассматриваемых объектов.

1) $A \cap B \Leftrightarrow a \wedge b$

2) $A \cup B \Leftrightarrow a \vee b$

3) $A \subseteq B \Leftrightarrow a \rightarrow b$

Введение. Анализ следствия на примере обихих фраз:

Если погода хорошая, и Дима выучил уроки, то Диме можно поиграть

в футбол, а иначе нельзя

Дима не учил уроки $\Rightarrow$ у мамы плохое настроение

Сейчас Дима играет в футбол. Верно ли, что у мамы хорошее настроение?

Пусть p - погода (хорошая - 1; плохая - 0); u - уроки (1 - сделаны; 0 - нет);

f - футбол (1 - играет; 0 - нет); n - настроение (1 - хорошее, 0 - нет)

Запишем на языке алгебры логики:

$p \wedge u \rightarrow f$; $\bar{u} \rightarrow n$, f . Верно ли n ?

Ответ: нет; Пример: $n=0$; $f=1$; p и u - любые.

р.с. функциями склади
sad story

Перепишем условие:

Погода хорошая, и Дима выучил уроки $\Leftrightarrow$ Диме разрешают играть в футбол

Дима не учил уроки $\Leftrightarrow$ у мамы портится настроение

$\begin{cases} f \Leftrightarrow \text{или} \\ u \Leftrightarrow \text{и} \\ f \end{cases}$ Тогда $f = u = \text{и} = \text{или} = 1$

Вывод: в повседневной речи может быть много слов, надо формализовать, т.к. нельзя сделать нормальное следствие.

Опр Пусть f - функция алгебры логики; Γ - некоторое множество функций алгебры логики, f называют семантическим следствием Γ и обозначают $\Gamma \models f$, если f истинно на всех наборах значений переменных, на которых истинны все функции Γ .

Пример: $(A; A \rightarrow B) \models B$

Опр Формальная система - набор $(\Sigma; F; A; R)$.

Σ - алфавит, счётное множество (из алфавита строят слова)

F - формулы (счётное множество конечных слов)

$F \subseteq \Sigma^*$ (Σ^* - множество конечных слов)

Пустое слово обычно обозначают ϵ .

A - аксиомы; $A \subseteq F$

синтаксическое следствие

R - правила вывода $((f_1, f_2, \dots, f_k) \vdash f_{k+1}) : F^k \rightarrow F$

Опр Формальная система исчисления высказываний (синтаксис алгебры логики):

выберем $(\neg, \rightarrow)$ в качестве логических операций (полная система)

$\Sigma = \{ \neg, \rightarrow, (,) \}$; счётное множество переменных $(x_1, x_2, \dots)$

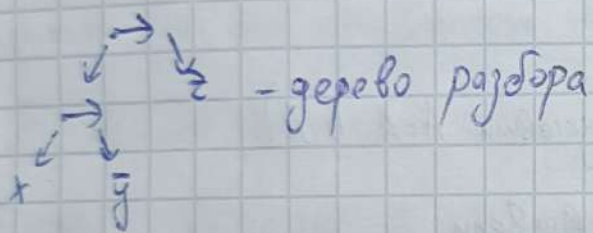
F: 1) x - формула, если x - переменная

2) $(\neg x)$ - формула, если x - формула

3) $(x \rightarrow y)$ - формула, если x, y - формулы

4) Других формул нет

$((x \rightarrow (\neg y)) \rightarrow z)$



Если дерево разбора строится, то формула, чиста - нет.

A: счётное множество аксиом

Схемы аксиом:

A, B и C - формулы

1) $(A \rightarrow (B \rightarrow A))$

2) $((A \rightarrow (B \rightarrow C)) \rightarrow (A \rightarrow B) \rightarrow (A \rightarrow C))$

3) $((\neg B) \rightarrow (\neg A)) \rightarrow (((\neg B) \rightarrow A) \rightarrow A)$

R: $A, A \rightarrow B \vdash B$

f выводится в исчислении высказываний, если $\exists (f_1; f_2 \dots f_n); n \in \mathbb{N}$ и $f_n = f$; f_i - аксиома, или $\exists j, k < i: f_j = (f_k \rightarrow f_i)$, n - длина вывода. Выведем $(A \rightarrow A)$:

1) Берём схему 2: $A = A$; $B = (A \rightarrow A)$; $C = A$

$f_1: ((A \rightarrow ((A \rightarrow A) \rightarrow A)) \rightarrow ((A \rightarrow (A \rightarrow A)) \rightarrow (A \rightarrow A)))$ - аксиома

2) Берём схему 1: $A = A$; $B = A \rightarrow A$:

$f_2: (A \rightarrow ((A \rightarrow A) \rightarrow A))$ - аксиома

3) $mp(f_1, f_2): \{f_1, f_2\} \vdash f_3: ((A \rightarrow (A \rightarrow A)) \rightarrow (A \rightarrow A))$

$\downarrow$
modus ponens - правило вывода

4) Берём схему 1: $f_4: (A \rightarrow (A \rightarrow A))$ (подставили $A = A$, $B = A$)

5) $mp(f_3, f_4): (f_3, f_4) \vdash f_5: (A \rightarrow A)$ - вывели

Утв Исчисление высказываний полно и корректно.

Опр f ^{условно} выводится из гипотез Γ , если $\exists$ цепочка $(f_1; f_2 \dots f_n) \vdash f$, где f_i - аксиома или гипотеза.

Теорема (критерий условной выводимости)

$$\Gamma \vdash f \Leftrightarrow \Gamma \models f$$

□ Доказательства нет, выдыхаем: h) ■

Лекция 4

Исчисление предикатов: алфавит и формулы. Сигнатура

Исчисление высказываний неудобно, т.к. можно вывести только тавтологии на множестве булевых переменных и функций.

Зададим формальную систему - исчисление предикатов:

Алфавит (Σ):

- 1) Все символы из ИВ для записи формул: $\{ (,); \rightarrow; \neg \}$
- 2) X - не более ^{тем} счётное множество символов переменных. Переменные могут принимать значения произвольного множества M .
- 3) F - не более тем счётное множество функциональных символов. Функции из F задают отображение $M^k \rightarrow M$, где $k \in \mathbb{N} \cup \{0\}$, т.е. функции дают в ответе элементы множества M . Если $k=0$, то под такой функцией подразумевается константа $c \in M$.
- 4) P - не более тем счётное множество предикатных символов. Предикаты из P задают отображение $M^k \rightarrow \text{bool}$, где $k \in \mathbb{N} \cup \{0\}$, т.е. функ предикаты возвращают true или false.
- 5) $;$ - разделительный символ
- 6) $\forall$ - квантор всеобщности

Формулы (F).

Все формулы - выражения, возвращающие bool.

Опр Терм:

- 1) Символ переменных
- 2) Если $f \in F_k$; $t_1, t_2, \dots, t_k$ - термы, то $f(t_1, t_2, \dots, t_k)$ - терм

Опр Элементарные формулы:

Если $A \in P_m$; $t_1, t_2, \dots, t_m$ - термы, то $A(t_1, t_2, \dots, t_m)$ - элементарная формула

Опр Формула:

- 1) Если A - элементарная формула, то A - формула
- 2) $(\neg A)$ - формула, если A - формула
- 3) $(A \rightarrow B)$ - формула, если A и B - формулы
- 4) $(\forall x A)$ - формула, если $x \in X$ и A - формула

Опр Сигнатура - конечное количество предикатных и функциональных символов с указанием арности.

Пример 1 Выразим некоторые формулы в простейшей арифметике.

Сигнатура: $\{+, \odot, =, >, 0, 1\}$.

В качестве множества M берём $\mathbb{N} \cup \{0\}$.

$+$ и $\odot \in F_2$, т.е. являются функциональными символами арности 2 (принимают 2 переменных)

$=, > \in P_2$, т.е. предикаты арности 2

$0, 1 \in F_0$, т.е. функции arity 0.

Хотим записать утверждение $\forall x \in F_0, x \geq 0$ "0-наименьший элемент" на языке ИЛ.

$\vee$ -выражается через $\neg$ и $\rightarrow$ $\Rightarrow$ можно её использовать

Формула: $(\forall x (\odot(x; 0) \vee \ominus(x; 0)))$

Опр Модель:

- 1) Сигнатура
- 2) Область интерпретации (M)
- 3) Интерпретация символов сигнатуры (придаём функциям и предикатам смысл),

переход от синтаксиса к семантике.

Пример 2

Работаем в той же сигнатуре арифметики.

Выразим квантор существования:

$$(\exists x B) = (\neg(\forall x(\neg B)))$$

Выразим $A_0(x)$ - предикат 0, т.е. $A_0(0) = \text{true}$, иначе false:

$$A_0(x) = \forall y (\ominus(y, \odot(x, y)))$$

Выразим $A_1(x)$ - предикат 1:

$$A_1(x) = \forall y (\ominus(y, \odot(x, y)))$$

Выразим $B_2(x, y)$ - предикат кратности, т.е. $x \div y$:

$$B_2(x, y) = \exists c (\ominus(x, \odot(y, c)))$$

Выразим формулу: "x-степень 2".

x-степень 2 $\Leftrightarrow$ Если $x \vdash t$, то $t=1$ или $t \vdash 2$. Тогда,

$$\text{"x-степень 2"} = \forall t (B_2(x, t) \rightarrow (A_1(t) \vee B_2(t, 2)))$$

Опр Теория - это набор замкнутых формул в данном языке.

Опр Оценка - это функция, которая сопоставляет свободным переменным в формуле конкретные значения из некоторой модели.

Лекция 5

Исчисление предикатов: аксиомы и правила вывода. Метод автоморфизмов.

Метод резолюции.

Пусть есть модель $(\mathbb{Q}; \otimes; \oplus, 0)$. Мы хотим выразить $(x=1)$

Но будто в данной модели 1 ниже не применима, выразить не получается. Sad story: (

Хотим научиться понимать, можно ли выразить формулу в заданной модели.

Опр Автоморфизм модели - биекция $\varphi: M \rightarrow M$ такая, что если возьмём

$\forall$ функцию f , то выполняется $\varphi(f(t_1; t_2; \dots; t_n)) = f(\varphi(t_1); \varphi(t_2); \dots; \varphi(t_n))$,

а если $\forall$ предикат P , то выполняется $P(\varphi(t_1); \varphi(t_2); \dots; \varphi(t_n)) = P(t_1; t_2; \dots; t_n)$.

Утв (метод автоморфизмов)

Если задан автоморфизм модели φ , то всё, что можно выразить в рамках

модели будет удовлетворять автоморфизму φ .

Вернёмся к модели $(\mathbb{Q}; \otimes; \oplus, 0)$.

Зададим отображение $\varphi: x \mapsto 2x$. Докажем, что φ -автоморфизм.

1) φ -биекция

2) $x \otimes y \Rightarrow 2x \otimes 2y \Rightarrow$ предикат $\otimes$ удовлетворяет автоморфизму

3) $\varphi(\oplus(x; y)) = \varphi(x \oplus y) = 2(x \oplus y)$

4) $\oplus(\varphi(x); \varphi(y)) = \oplus(2x; 2y) = 2x \oplus 2y$ } $\Leftrightarrow \Rightarrow$ функция $\oplus$ удовлетворяет

4) $\varphi(0) = 0 \Rightarrow 0$ удовлетворяет

$\Downarrow$

φ - автоморфизм

Пусть $(x=1) = I(x)$.

$I(1) = \text{true}$

$I(\varphi(1)) = I(2) = \text{false}$

$I(1) \neq I(\varphi(1)) \Rightarrow I$ - не подпадает под автоморфизм $\Rightarrow I$ - невыразима

Аксиомы исчисления предикатов (А):

1) $(A \rightarrow (B \rightarrow A))$

2) $(A \rightarrow (B \rightarrow C)) \rightarrow ((A \rightarrow B) \rightarrow (A \rightarrow C))$

3) $((\neg B) \rightarrow (\neg A)) \rightarrow ((\neg B) \rightarrow A \rightarrow B)$

} аксиомы из ИВ

4) $\forall x (A \rightarrow B) \rightarrow (A \rightarrow \forall x B)$, если в A нет свободных вхождений x .

5) $\forall x (A \rightarrow A(x := t))$

~~можно заменить переменные, если нет свободных вхождений~~
при выполнении $\odot$ (см. сл. стр.) $\rightarrow$

Правила вывода (R):

1) $A \rightarrow B; A \vdash B$ - из ИВ

2) $A \vdash \forall x A$

Опр Формула в ИП называется общезначимой, если она истинна в любой интерпретации.

Опр Пусть дана формула $\forall x A$. Тогда A - область действия квантора по x .

Опр Все вхождения x в A называются связанными. Если вхождение переменной не лежит в области действия квантора по данной переменной, то

такое вхождение называют свободным.

Пример

$$\forall x (x+y < 0)$$

↑ ↓
связанное свободное
вхождение вхождение

* t можно подставлять только если ни одно свободное вхождение x в A не лежит в области действия квантора по переменным, входящим в t ; t -терм.

Утв

Если φ -тавтология, то $\bar{\varphi} = 0$. φ -формула в виде КНФ. Хотим понять, выполняется ли она

Утв (метод резолюции).

Пусть $\varphi = (a_1 \vee \bar{a}_2 \vee a_3) \wedge (\bar{a}_1 \vee a_4 \vee \bar{a}_5)$

Тогда $\mathcal{D}_1 = (a_1, \bar{a}_2, a_3)$; $\mathcal{D}_2 = (\bar{a}_1, a_4, \bar{a}_5)$ - обозначим дизъюнкты.

Р-м переменной. Если есть только a в КНФ и нет $\bar{a}$, то $a=1$ - неинтересно

Аналогично наоборот.

Тогда рассмотрим две такие скобки: $(a \vee A)$ и $(\bar{a} \vee B)$

↓ ↓
 $\{a\} \cup \mathcal{D}_A$ $\{\bar{a}\} \cup \mathcal{D}_B$

Опр Резольвентой дизъюнктов $a \vee A$ и $\bar{a} \vee B$ называется $A \vee B$ или же $\mathcal{D}_A \cup \mathcal{D}_B$

Утв Резольвента - семантическое следствие φ (на этом моменте попросили не учитывать)

Перепишем все возможные резольвенты. Повторяем до тех пор, пока не получим полный набор резольвент.

XIV Формула выполнима $\Leftrightarrow$ Полный набор резольвент не содержит пустой дизъюнкции.

Лекция 6

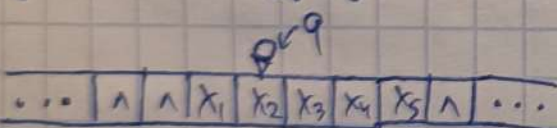
Машины Тьюринга. Определение конфигурации.

Опр Машина Тьюринга - это набор $(A; Q; Q_f; \delta; q_0; \Lambda)$, где:

- A - конечное множество, называемое алфавитом МТ
- Q - конечное множество, называемое множеством состояний МТ. $A \cap Q = \emptyset$
- Q_f - множество финальных состояний МТ. $Q_f \subseteq Q$
- δ - функция (таблица) переходов. $\delta: A \times Q \rightarrow A \times \{-1, 0, 1\} \times Q$
- q_0 - начальное состояние; $q_0 \in Q$
- Λ - пустой символ; $\Lambda \in A$

Пояснения, как это выглядит.

Есть память в виде бесконечной ленты с ячейками. В ячейках написаны символы из алфавита МТ. Над какой-то ячейкой находится считывающее устройство. Визуализация:



Машина Тьюринга получает на вход слово x (конкретную ленту с символами и положением считывающего устройства). Совершает действия над лентой и на выход даёт некое слово y .

Опр Конкретная лента с памятью + положение считывающего устройства называется конфигурацией Машины Тьюринга. (неформальное)

Опр Конфигурацией МТ $(A; Q, Q_f; \delta; q_0; \lambda)$ называется слово w в алфавите $A \cup Q$, которое содержит ровно один символ из множества Q , а первый и последний символы w не являются пустыми. (формально) Начальная конфигурация МТ имеет вид $q_0 u$, где $u \in (A \setminus \epsilon)^*$ - конечное слово, которое называется входом МТ.

Пояснения, как это работает:

Любое действие МТ - отображение $Q \times A \rightarrow A \times \{-1; 0; 1\} \times Q$.

Сначала есть ячейка, на которую указывает считывающее устройство, какой-то символ $a_1 \in A$ и состояние $q_1 \in Q$. За ход мы заменяем символ a_1 на какой символ $a_2 \in A$ (возможно $a_1 = a_2$), меняем состояние q_1 на q_2 (возможно $q_1 = q_2$) и делаем сдвиг считывающего устройства на 1 влево или вправо (-1 или 1), либо остаемся в той же ячейке.

Пример хода:

$\delta(a, q_1) = (b, +1; q_2)$

q_1	a	?	...
-------	-----	---	-----

→

q_2	b	?	...
-------	-----	---	-----

Считаем, что если МТ приходит в одно из финальных состояний, то она сразу останавливает работу.

Пример конфигурации:

...	^	a	b	c	a	b	^	...
-----	---	---	---	---	---	---	---	-----

$\Rightarrow$ $abcbab$

$\underset{q_1}{\uparrow}$

9

e)

1

$$A = \{1, 1, \# \}$$
$$Q = \{q_0, q_1, q_f, q_{er}\}$$
$$Q_f = \{q_f; q_{er}\}$$

Q_f - финальное состояние при успешном выполнении

σ_{er} - финальное состояние, выкидываемая ошибка

1

1

1

(Пример для конкретных чисел, программа работает в общем случае)

$$\delta:$$

Q \ A	1	0	Λ
q_0	$(1; +1; q_0)$	$(0; +1; q_0)$	$(\Lambda; +1; q_1)$
q_1	$(0; 0; q_1)$	$(1; -1; q_1)$	$(\Lambda; 0; q_{er})$

$$A = \{1, 0, \Lambda\}$$

$$Q = \{q_0, q_1, q_e, q_{er}\}$$

$$Q_e = \{q_e, q_{er}\}$$

Утв Множество всех Машии Тьюринга счётно.

Опр Расширенной конфигурацией МТ назовём двустороннюю бесконечную последовательность $\dots \Lambda w \Lambda \dots$, где w - конфигурация МТ, а на месте многоточий стоят пустые символы Λ .

Отображение на расширенных конфигурациях МТ:

$$d_M(uqv) = \begin{cases} u a' q' v' ; & \delta(a; q) = (a'; +1; q') \\ u q' a' v' ; & \delta(a; q) = (a'; 0; q') \\ u' q' b a' v' ; & \delta(a; q) = (a'; -1; q') \end{cases}$$

где $v = a v'$; $u = u' b$

Отображение на конфигурациях формализует работу МТ за один такт.

Работа МТ M полностью задаётся начальной конфигурацией и отображением на конфигурациях и может быть описана конечной или бесконечной последовательностью конфигураций:

$$w_0; w_1 \dots w_n \dots, \text{ где } w_{i+1} = d_M(w_i)$$

Опр Последовательность $w_0, w_1, \dots, w_n, \dots$ называется протоколом работы МТ.

Опр Результатом работы МТ является слово, которое получается из последней конфигурации вычеркиванием символа состояния головки.

Кодирование МТ:

Берём конечный алфавит: $A = \{a_1, \dots, a_n\}$; $|A| \in \mathbb{N}$

Конечное множество состояний: $Q = \{q_0, q_1, \dots, q_m\}$, $|Q| \in \mathbb{N}$

Закодируем МТ следующим образом,

$q_0 q_1 \dots q_m \# a_1 a_2 \dots a_n \# q_{k_1} q_{k_2} \dots q_{k_k} \# a_0 0 + 1 q_1 \# \dots \# \dots$
 $\downarrow$ $\downarrow$ $\downarrow$ $\downarrow$
 Q раздиратель A Q_F один переход из δ

4) нас пришлось задать МТ полностью на конечной ленте.

Лекция 7

Вычислимость и разрешимость. Другие варианты МТ.

Кодирование и моделирование. Универсальная МТ.

Опр Частично определённая функция $f: \mathbb{N}^n \rightarrow \mathbb{N}$ называется вычислимой на МТ, если $\exists$ такая МТ M , что для $\forall$ набора $(m_1, m_2, \dots, m_n)$, принадлежащего области определения f , работа M на входе $\# \underbrace{11\dots 1}_{m_1+1} \# \underbrace{11\dots 1}_{m_2+1} \# \dots \# \underbrace{11\dots 1}_{m_n+1}$ завершится результатом

$$\underbrace{11\dots 1}_{f(m_1, m_2, \dots, m_n)+1}$$

Опр Решающая машина Тьюринга имеет ровно 2 финальных состояния: q_{yes} и q_{no} .

Опр Язык L в конечном алфавите A называется разрешимым, если $\exists$ такая решающая МТ с алфавитом $B \supseteq A$, которая на $\forall$ входе $w \in L$ останавливается в состоянии q_{yes} , а на $\forall$ входе $w \notin L$ останавливается в состоянии q_{no} . При этом пустой символ $\lambda \in A$.

Опр Характеристическая функция χ_L языка L :

$$\chi_L = \begin{cases} 1; & w \in L \\ 0; & w \notin L \end{cases}$$

Утв L - разрешим $\Leftrightarrow \chi_L$ - вычислима

Другие варианты МТ:

- Недетерминированная МТ;

$$\delta: A \times Q \rightarrow 2^{A \times \{-1, 0, 1\} \times Q}$$

Пример: $\delta(a, q) = \{(b, +1, q_1), (a, 0, q_2)\}$

В недетерминированной МТ комбинация символа на ленте и текущего состояния может допускать одновременно параллельно несколько переходов в следующие состояния, то есть она может одновременно находиться во многих состояниях.

- МТ с лентой, бесконечной в одну сторону.
- К-ленточная МТ, где $K \in \mathbb{N}$

К-ленточная МТ не очень удобна в записи конфигураций. В ней мы записываем состояния всех лент подряд, но удобнее записывать их отдельно.

Утв Если язык разрешим/функция вычислима на какой-то МТ, то это выполняется на $\forall$ МТ.

Опр МТ M' моделирует работу МТ M , если она удовлетворяет следующему свойству: на входе вида $s(w)$, где w - вход M , результатом работы M' является $s(u)$, где u - результат работы M на входе w . Здесь $s(x)$ - код слова x , то есть отображение из множества слов над алфавитом M в множество слов над алфавитом M' , позволяющее по $s(x)$ однозначно восстановить x .

Утв Для $\forall$ МТ $M \exists$ МТ в двухбуквенном алфавите M' , моделирующая работу M .

Утв Для $\forall$ k -ленточной МТ $M \exists$ одноленточная МТ M' , моделирующая работу M .

Опр Универсальной машиной Тьюринга называют такую МТ, которая моделирует работу $\forall$ другой МТ. Входом такой универсальной МТ является описание МТ и ее' входа.

Утв $\exists$ универсальная МТ с одной лентой и двухбуквенным алфавитом.

Лекция 8

Алгоритмически неразрешимые проблемы

1) Проблема самоприменимости

Множество алгоритмов счётно. Зафиксируем их нумерацию $A_0, A_1, \dots$ с возможными повторениями.

Число входов тоже счётно, поэтому без ограничения общности, считаем, что на вход подаётся натуральное число.

Определим функцию $F: \mathbb{N} \rightarrow \mathbb{N}$ таким образом:

$$\begin{cases} F(i) = 1, & \text{если } A_i \text{ останавливается на входе } i. \\ F(i) = 0, & \text{иначе} \end{cases}$$

Проблема самоприменимости состоит в вычислении функции F .

Утв F алгоритмически неразрешима при $\forall$ нумерации алгоритмов.

□ Предположим противное: F -вычислима. Тогда $\exists$ МТ M_F , которая вычисляет F .

Пусть $M_F(n)$ - её результат на входе n .

Построим МТ M'_F так:

$$M'_F \Rightarrow \begin{cases} \text{пишет 1 и останавливается, если } M_F(n) = 0 \\ \text{зацикливается, если } M_F(n) = 1. \end{cases}$$

Т.к. M'_F - МТ, то у её алгоритма есть какой-то номер m .

Запустим $M'_F(m)$.

1) Пусть $M'_F(m)$ остановилась, тогда $F(m) = 0$, то есть МТ M'_F не остановилась на входе m .

Противоречие.

2) Пусть $M'(m)$ закинула. Тогда $f(m)=1$, и M' должна остановиться.

Противоречие

⇓

F - невычислима



2) Проблема остановки

Определим функцию $f_s: \mathbb{N} \rightarrow \mathbb{N}$ таким образом:

$$f_s(i, j) = \begin{cases} 1; & \text{Аи останавливается на входе } j \\ 0; & \text{иначе} \end{cases}$$

Проблема остановки заключается в вычислении функции f_s .

Ув f_s неразрешима при $\forall$ нумерации алгоритмов.

□ Сначала построим МТ M_w , которая делает следующее:

- 1) С пустого входа печатает w
- 2) Смещается на первый символ
- 3) Запускает M на w .

Построение M_w :

$$\delta(q_0, \lambda) = (w_1, +1; q_1)$$

$$\delta(q_1, \lambda) = (w_2, +1; q_2)$$

⋮

$$\delta(q_{|w|+1}, \lambda) = (w_{|w|}, 0; q_n)$$

$$\delta(q_n, a) = (a, -1; q_n)$$

$$\delta(q_n, \lambda) = (\lambda, +1; q_{0M})$$

Здесь w_i - i -ый символ слова w

a - $\forall$ непустой символ

q_{0M} - начальное состояние M . Дальше выполняется ее код

Построение M_w свело задачу к вопросу: „остановится ли MT на пустом входе?“

Возьмём произвольную нумерацию алгоритмов $A_0, A_1, \dots$

Пронумеруем входы так, что у A_i , i -ый вход — пустой.

Тогда задача свелась к тому, что спрашивается, остановится ли A_i на входе i .

Это неразрешимая проблема самоприменимости для $\forall$ нумерации алгоритмов.

Тогда f_5 неразрешима, т.к. зная её вычислить, мы бы умели решать проблему самоприменимости. ■

Опр L_{stop} — язык в алфавите $\{0,1\}$, состоящий из тех описаний $\langle M, w \rangle$

(MT M и входного слова w), для которых M останавливается на входе w .

Утв Язык L_{stop} неразрешим

3) Достижимость в ассоциативном исчислении

Заданы правила преобразования слов в алфавите $A: L_i \rightarrow R_i \in A^*$ (о нет, отсылка на :
звездочка из алфавита :D)

$L_1 \rightarrow R_1, L_2 \rightarrow R_2, \dots, L_n \rightarrow R_n$, то есть $\forall$ вхождение L_i в слово можно заменить на R_i .

Задача достижимости: по двум заданным словам и набору правил определить, можно ли одно слово получить из другого заданными преобразованиями.

Утв Задача достижимости неразрешима

□ Построим M' так. Пусть M — MT , решающая задачу достижимости.

1) Работает как M

2) Из любого финального состояния M делает следующее: стирает всё с ленты

Лажа, но
вроде на эже
не надо

(+GarbageCollector; java moment) и переходит в своё единственное финальное состояние и останавливается.

Тогда если бы эта задача была разрешима, то про M' можно было бы всегда сказать, останавливается ли она на входе w , то есть была бы разрешима проблема останова. $\Rightarrow$ задача достижимости неразрешима $\blacksquare$

4) Равенство слов в полугруппе

Заданы правила преобразования слов в алфавите A : $L_i, R_i \in A^*$ (окрестность...):

$L_1 \leftrightarrow R_1, L_2 \leftrightarrow R_2 \dots, L_n \leftrightarrow R_n$, то есть $\forall$ вхождение L_i в слово можно заменить на R_i и наоборот.

Задача достижимости: по двум заданным словам и набору правил определить, можно ли одно слово из другого заданными преобразованиями.

Утв Эта задача достижимости неразрешима.

$\square$ Сделаем $2n$ односторонних замен: $L_1 \rightarrow R_1, L_2 \rightarrow R_2 \dots L_n \rightarrow R_n; R_1 \rightarrow L_1 \dots R_n \rightarrow L_n$

Получили произвольную задачу $\blacksquare$

5) Проблема соответствий Поста

Системой соответствий Поста называется набор пар слов в конечном алфавите

$$A : (u_1, v_1), (u_2, v_2) \dots (u_n, v_n)$$

Решением системы соответствий называется такая конечная последовательность

$$i_1, i_2 \dots i_k, \text{ что } u_{i_1} u_{i_2} \dots u_{i_k} = v_{i_1} v_{i_2} \dots v_{i_k}$$

УТВ Задача о $\exists$ решения системы соответствий Поста неразрешима

б) Функция трудоемкости $R_{\text{Рого}}$

Функция трудоемкости $R_{\text{Рого}}$ $R(n)$ определяется как максимальное количество единиц, которое может оказаться на ленте, в результате работы МТ с n состояниями на пустом входе и алфавитом $\{1\}$ при условии, что МТ останавливается на пустом входе.

Вычислим $R(1)$ и $R(2)$:

$R(1) = 0$, потому что $q_0 = q_f$, и МТ сразу остановилась.

$R(2) = 1$:

1) $(q_0, 1) = (q_i, \forall i \neq 0)$ - там одна "1" и сразу останавливается

2) $(q_0, 1) = (q_0, 1; \forall)$ - зациклилась, невозможно

$(q_0, 1; \pm 1)$ - зациклилась, невозможно

$(q_0, 1; 0)$ - единственный вариант. Появилась одна "1"

3) $(q_0, 1) = (q_0, \forall; \forall)$ - зациклилась, т.к. никогда не перейдет в q_f

$(q_f, 1; \forall)$ - стало 0 "1"

$(q_f, 1; \forall)$ - осталась 1 "1" $\Rightarrow$

$\Rightarrow R(2) = 1$. (*) - см. в конце

Есть оценка $R(3k+3) \geq 4k$, т.е. $\exists$ МТ M_{3k+3} , которая с $3k+3$ состояниями выводит $4k$ "1". (её нам не представили, просто возьмём готовую, поверив в её существование)

Утв $R(n)$ невычислима

□ Предположим противное. Тогда $\exists$ МТ M_R , которая её вычисляет. Тогда на вход она получает $\underbrace{1 \dots 1}_n$, а выводит $\underbrace{1 \dots 1}_{R(n)}$. Пусть $c \in \mathbb{N}$ — количество

состояний M_R .

Возьмём следующую МТ M' :

1) Она с пустого входа работает, как M_{3k+3} и печатает на ленте $4k$ „1“.

2) Используем $3k+3$ состояния.

3) Переходим на первую „1“, не меняя ленты (хватит 3 состояния)

4) Запускаем M_R . То есть в ответе ждём $R(4k)$

Всего состояний $3k+6+c$. При этом мы вывели $R(4k) \Rightarrow R(3k+6+c) \geq R(4k)$.

Т.к. $R(n)$ строго монотонно возрастает, то $4k \leq 3k+6+c$, что неверно при больших k . Противоречие. $\Rightarrow R(n)$ — невычислима ■

P.S. $\forall$ — квантор очевидности

* Приведём алгоритм для МТ, которая по $3k+1$ состояний пишет $4k$ „1“.

($3k+3$ тогда тоже пишется):

$$\delta(1; q_{3i+0}) = (1; +1; q_{3i+1})$$

$$\delta(1; q_{3i+0}) = (1; +1; q_{3i+0})$$

$$\delta(1; q_{3i+1}) = (1; -1; q_{3i+2})$$

$$\delta(1; q_{3i+1}) = (1; +1; q_{3i+2})$$

$$\delta(1; q_{3i+2}) = (1; +1; q_{3i+1})$$

$$\delta(1; q_{3i+2}) = (1; +1; q_{3i+3})$$

Лекция 9

Разрешимость, полурешимость, перечислимость. m-сводимость

Разрешимые языки замкнуты относительно:

- 1) Объединения
- 2) Пересечения
- 3) Конкатенации
- 4) Дополнения
- 5) Декартова произведения
- 6) Итерации

Пусть есть язык L и $n \in \mathbb{N}$. Итерацией L назовём:

$$L^* = \{ \varepsilon, UL, UL^2, U \dots UL^n, U \dots, L^\infty \}$$

$x \in L^* \Leftrightarrow \exists$ разбиение x на подстроки $x_1, x_2, \dots, x_n$ такие, что $x_i \in L \quad \forall i \in \overline{1, n}$ или ε .

Утв Разрешимые языки замкнуты относительно итерации

□ Пусть на вход дано слово x . Тогда мы разбиваем не более чем на $|x|$ подстрок. Всего так $2^{|x|-1}$ разбиений $\Rightarrow$ надо конечное число действий, чтобы определить $x \in L^*$ или нет $\Rightarrow L^*$ - разрешим $\blacksquare$

Утв Конечные языки разрешимы

□ См №3 из гл 3 $\blacksquare$

Опр язык L называется полурешимым (допустимым), если $\exists$ такая МТМ, что:

- 1) на $\forall$ входе $w \in L$ М останавливается в состоянии q_{yes}
- 2) на $\forall$ входе $w \notin L$ М не останавливается в состоянии q_{yes} (может остановиться в q_{no} или закрутиться)

Утв Пусть язык L - полурешим. Язык $\bar{L}$ - дополнение L тоже полурешим $\Rightarrow L$ - разрешим.

$$\square \begin{array}{l} L\text{-полурешим} \Rightarrow \exists \text{ МТМ: } \forall w \in L \rightarrow q_{yes} \\ \bar{L}\text{-полурешим} \Rightarrow \exists \text{ МТМ: } \forall w \notin L \rightarrow q_{yes} \end{array} \Rightarrow$$

$\Rightarrow$ запустим одновременно М и $\bar{M}$. Одна из них остановится и выдаст ответ $\Rightarrow$

$\forall w$ можно определить, лежит ли оно в $L \Rightarrow L$ - разрешим $\blacksquare$

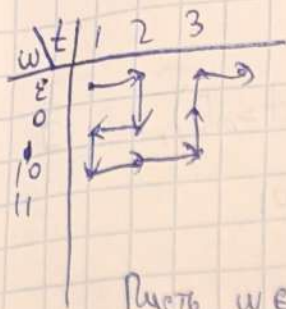
Опр Язык L называется перечислим, если $\exists$ МТ, которая, работая с пустого входа напечатает на выходной ленте все слова языка L , разделяя их специальными символами. При этом МТ никогда не изменяет непустые символы выходной ленты.

Язык L может быть бесконечным. Имеется в виду, что каждое слово языка появляется на ленте за конечное время.

Утв Язык L - полурешим $\Leftrightarrow L$ - перечислим

$\square \Rightarrow$ Надо построить МТ М, которая перечислит М. Построим следующую таблицу:

МТМ,



По горизонталям стоят все слова ~~языка~~ в лексикографическом порядке. По вертикали количество тактов.

Идем змейкой, и если слово есть в L , то выводим его.

то -

МТМ

Пусть $w \in L$. Т.к. L - полурешим, то МТМ выведет q_{yes} на нём и остановится за конечное число тактов m . Клетку $(m; w)$ мы когда-то посетим $\Rightarrow$ слово w выведем $\Rightarrow L$ - разрешим

$\Leftarrow$ L - разрешим. Тогда $\forall w \in L$ будет когда-то выведено. Будем перебирать все слова в порядке вывода. Т.к. они все $\in L$, то МТМ от них остановится и выдаст $q_{yes} \Rightarrow \forall$ слово $w \in L$ можно проверить на МТМ, если оно $\in L \Rightarrow L$ - полурешим $\blacksquare$

Утв Множество разрешимо, если оно есть область определения вычислимой функции

Утв Множество разрешимо, если оно есть область значений вычислимой функции.

Утв Если язык и его дополнение разрешимы, то язык разрешим.

$\square$ L - разрешим $\Leftrightarrow L$ - полурешим. Уже доказано ранее $\blacksquare$

Разрешимые языки замкнуты относительно:

- 1) Объединения
- 2) Пересечения
- 3) Конкатенации
- 4) Декартова произведения
- 5) Итерации

УТВ Перечислимые языки не замкнуты относительно дополнения

Опр Язык $A \in \mathcal{L}$ m -сводится к языку $B \in \mathcal{L}$ (обозначается $A \leq_m B$), если $\exists$

всюду определённая вычислимая функция $f: \mathbb{N} \rightarrow \mathbb{N}$ такая, что:

$$x \in A \Leftrightarrow f(x) \in B$$

Свойства m -сводимости:

- 1) $A \leq_m A$ (рефлексивность)
- 2) $A \leq_m B; B \leq_m C \Rightarrow A \leq_m C$ (транзитивность)
- 3) $A \leq_m B \Rightarrow \mathbb{N} \setminus A \leq_m \mathbb{N} \setminus B$

Если $A \leq_m B$, то:

- 1) B -разрешим $\Rightarrow A$ -разрешим
- 2) B -перечислим $\Rightarrow A$ -перечислим

Лекция 10

Теорема Райса. Теорема Тарского. Теорема Гёделя

Пусть $\mathcal{A}$ (А красивое) — некоторое нетривиальное семейство вычислимых функций, т.е. $\exists$ такие вычислимые функции f_1 и f_2 такие, что $f_1 \in \mathcal{A}$; $f_2 \notin \mathcal{A}$.

Теорема (Райса) Алгоритмически неразрешима следующая проблема;

По описанию МТ M решить, принадлежит ли вычисляемая M функция f семейству $\mathcal{A}$.

Доказательство см. в конце

□ Р-и 2 множества:

$A = \{M \mid M \text{ останавливается на } e\}$

$B = \{M \mid M \text{ реализует функцию } f \in \mathcal{A}\}$

По условию $\exists$ функции $f_0 \notin \mathcal{A}$ и $f_1 \in \mathcal{A}$.

Пусть есть код x МТ M , и мы хотим понять $x \in A$. Хотим построить φ — вычисляющую функцию, преобразующую x в $\varphi(x)$, чтобы был вопрос $\varphi(x) \in B$,

т.е. построить φ — значит построить m -сводимость.

Опишем МТ, вычисляющую $\varphi(x)$:

На входной ленте (I) изначально слово x .

На ленте II будет новый код.

Действия:

1) $\varphi(x)$ печатает на II ленту код МТ, переносащий входное слово и на

III Ленту и стирает всё с ленты I. Заканчивает в $\tilde{q}_0$.

2) На ленте II печатает код x , заменяя q_0 на $\tilde{q}_0$.

3) Из всех финальных состояний MT_x переходит в q_{01} .

4) Из q_{01} запускает код MT_i (которая вычисляет f_i), работает на ленте II.

Тогда $\varphi(x)$ вычисляет f_0 или f_i . Это зависит от того, останавливается ли x на пустом месте $\Rightarrow$ m -символ к stop $\Rightarrow$ неразрешима 

Р-м исчисление предикатов:

1) Множество формул - разрешимо

2) Множество аксиом - разрешимо

3) Множество правил вывода - разрешимо

4) Множество выводов - разрешимо

5) Множество общезначимых формул - неразрешимо, но не разрешимо.

Утв Для произвольной ФС: Если множества аксиом и правил вывода разрешимы, то множество выводимых в этой ФС формул разрешимо.

Утв Из неразрешимости множества формул следует невозможность построения формальной системы с разрешимыми множествами аксиом и правил вывода, которая выводила бы в точности формулы из этого множества.

Стандартная модель формальной арифметики: $(\mathbb{N}; 0; E; \forall; +; \times)$
 ↓ равенство ↓ следующий

Теорема (Тарского) Множество формул формальной арифметики, истинных в стандартной интерпретации, неперечислимо

Теорема (Геделя) Формальная арифметика неполна.

□ Рассмотрим 2 множества:

$$A = \{M \mid M \text{ останавливается на } \epsilon\}$$

$$B = \{M \mid M \text{ вычисляет } \varphi \in \mathcal{R}\}$$

Пусть f - нигде не определённая вычислимая функция (MT: while true)

Рассмотрим случаи:

1) $f \notin \mathcal{R}$. Тогда $\exists g \in \mathcal{R}$

Пусть дан код x МТМ. Построим вычислимую $\varphi(x)$ следующим образом:

- 1) Сохраним входное слово на другую ленту
- 2) Сотрём его с входной ленты
- 3) Запустим x на ϵ
- 4) Очистим ленту
- 5) Вернём входное слово на эту ленту
- 6) Запустим M_g , вычисляющую g .

Тогда $\varphi(x)$ - это МТ, которая вычисляет $g \Leftrightarrow x$ останавливается на $\epsilon \Rightarrow$

$\Rightarrow x \in A \Leftrightarrow \varphi(x) \in B \Rightarrow$ построили m -сводимость $A \leq_m B$. Т.к. A неразрешим,

то и B неразрешим

2) $f \in \mathcal{R} \Rightarrow f \notin \overline{\mathcal{R}}$. Проводим для $\overline{\mathcal{R}}$ рассуждения, аналогичные п.1. ■

Лекция II

Арифметическая иерархия. Универсальная функция. Универсальное

множество

Опр Графиком функции f с натуральными аргументами и значениями называется множество $F = \{ \langle x, y \rangle \mid \exists f(x) = y \}$

Утв Функция f вычислима $\Leftrightarrow$ её график F перечислим.

Опр Проекцией множества Q пар натуральных чисел называется множество P такое, что $x \in P \Leftrightarrow \exists y: \langle x, y \rangle \in Q$

Утв Множество P перечисливо $\Leftrightarrow \exists Q$ - разрешимое множество, такое, что

P - проекция Q

$\square \Rightarrow P$ -перечисливо $\Rightarrow \exists$ МТ M_P , его перечисляющая $\Rightarrow$ язык пар $\langle t; x \rangle$, где x появляется на ленте не более чем за t шагов, ~~перечислим~~ разрешим. Назовём это Q . Заметим, что P - проекция Q .

$\Rightarrow$ Пусть $Q(x, y)$ - разрешимое свойство, МТ M_Q - его разрешает.

Построим МТ M_P перечисляющую так. Упорядочим все слова в лексикографическом порядке. Будем идти по таблице змейкой.

	y_1	y_2	y_3	...
x_1	$\rightarrow$			
x_2	$\leftarrow$	$\rightarrow$		
x_3	$\leftarrow$	$\leftarrow$	$\rightarrow$	
$\vdots$				

На каждой паре (x_i, y_j) запустим M_Q . Она гарантированно выдаст ответ. Если $\langle x, y \rangle \in Q$, то выведем x . Таким образом, $\forall x$ появится на ленте за конечное время ■

Утв $\forall$ перечислимый язык P можно задать, как проекцию разрешимого множества Q , или же:

$$x \in P \Leftrightarrow \exists y: \langle x, y \rangle \in Q.$$

Назовём это Σ_1 .

Опр Языки, дополнение которых является перечислимым, называют коперечислимыми.

Утв $\forall$ коперечислимый язык L можно задать в виде:

$$x \in L \Leftrightarrow \forall y: \langle x, y \rangle \in Q_2, \text{ где } Q_2 - \text{ разрешимое множество.}$$

Назовём это Π_1 .

Опр Свойство (язык, множество) A принадлежит классу Σ_n , если его можно представить в виде $A(x) \Leftrightarrow \exists y_1 \forall y_2 \exists y_3 \dots ? y_n B(x, y_1, y_2, \dots, y_n)$, где B - разрешимое свойство. Кванторы чередуются. ? - какой-то квантор, зависит от чётности n .

Опр Свойство (язык, множество) A принадлежит классу Π_n , если его можно представить в виде $A(x) \Leftrightarrow \forall y_1 \exists y_2 \forall y_3 \dots ? y_n B(x, y_1, y_2, y_3, \dots, y_n)$, где B - разрешимое свойство. Кванторы чередуются.

Свойства классов Σ_n и Π_n :

- 1) $\Sigma_1 \cap \Pi_1$ - множество разрешимых языков
- 2) Σ_n и Π_n замкнуты относительно объединения и пересечения
- 3) Классы Σ_n и Π_n „наследственны вниз“ относительно т-сводится. Это значит:
 $A \leq_m B; B \in \Sigma_n(\Pi_n) \Rightarrow A \in \Sigma_n(\Pi_n)$

$$4) \Sigma_n \cup \Pi_n \subseteq \Sigma_{n+1} \cap \Pi_{n+1}$$

$$5) \Sigma_n \subseteq \Sigma_{n+1}; \Pi_n \subseteq \Pi_{n+1}$$

$$6) L \in \Sigma_n \Leftrightarrow \bar{L} \in \Pi_n$$

Опр Функция U двух натуральных аргументов называется универсальной для класса вычислимых функций одного аргумента, если $\forall n \in \mathbb{N}$ функция

$U_n: X \rightarrow U(n; x)$, т.е. сечение функции U при фиксированном n является вычислимой функцией, и все вычислимые функции встречаются среди U_n

Утв $\exists U(n; x)$ — вычислимая функция двух аргументов, являющаяся универсальной для класса вычислимых функций одного аргумента.

□ Машины Тьюринга, принимающих на вход один аргумент, счётно. Запишем коды всех таких МТ. Эти коды упорядочим по возрастанию. Так мы все МТ,

вычисляющие функции одного аргумента, присвоили вычислимый номер. Строим

МТ, которая по двум аргументам: номеру n и входу x запускает МТ с номером

n на входе x . Данная МТ вычисляет нужную нам универсальную функцию двух аргументов ■

Опр Множество $W \subseteq \mathbb{N} \times \mathbb{N}$ называется универсальным для некоторого класса множеств натуральных чисел, если все сечения $W_n = \{x \mid \langle n, x \rangle \in W\}$ мно-

жества W принадлежат этому классу и других множеств в этом классе нет.

Утв $\exists$ пересчитываемое множество пар натуральных чисел, универсальное для класса всех пересчитываемых множеств натуральных чисел.

□ Можно взять область определения универсальной функции для класса вычислимых. ■

Утв Для $\forall n \in \mathbb{N}$ в классе Σ_n $\exists$ множество, универсальное для всех множеств класса Σ_n . Его дополнение будет универсальным в классе Π_n .

Утв Универсальное $\Sigma_n(\Pi_n)$ множество не принадлежит классу $\Pi_n(\Sigma_n)$.

Лекция 12

Random Access Machine (RAM). Машина Минского.

Нормальные алгоритмы Маркова.

УТВ (Тезис Тьюринга) $\forall$ алгоритм может быть реализован в виде МТ.

Модели, вычисляемые процедурным способом:

- 1) Random Access Machine (RAM)
- 2) Машина Минского
- 3) Нормальные алгоритмы Маркова

Также есть функциональные определения алгоритма.

Random Access Machine (RAM).

Устройство:

- 1) Возможно обращаться к ячейкам памяти по их индексу.
- 2) Одна ячейка хранит целое число
- 3) Ячейки разделяются на 3 группы: вход, память и выход. В каждой группе ячейки заномерованы целыми неотрицательными числами
- 4) Кроме ячеек RAM включает в себя программу — список команд.

Команды:

$\langle \text{наименование команды} \rangle \langle \text{тип операнда} \rangle \langle \text{операнд} \rangle$

- 1) Арифметические операции (второй операнд берётся из ячейки номер 0)

2) Чтение с входной ленты

3) Запись на выходную ленту

Операнды:

Операнд - целое число. Типы операндов:

1) Константы (=)

Значение $v(i)$ константы i совпадает с ней самой

2) Переменные (λ)

Значение $v(i)$ переменной i равно числу, записанному в ячейке с номером i .

3) Указатели (*)

Значение $v(i)$ указателя i равно числу, записанному в ячейке с номером, который содержится в ячейке i .

Работа:

Начинается с исполнения первой команды.

Все команды делятся на 2 типа: операции и переходы.

После исполнения операции происходит выполнение следующей в списке команды.

Исполнение перехода определяет номер команды, которая должна исполниться следующей.

Работа заканчивается в одном из следующих случаев:

1) Исполнена последняя команда, и это операция.

2) Исполнена программа перехода, но полученное в результате неё число не является номером команды.

3) Очередная операция не может быть исполнена.

В начале работы ячейки RAM пусты кроме нескольких первых входных. Состояние RAM — это сетка $(n; x; r; y)$, где n — положительное целое число, счётчик команд, x, r, y — бесконечные последовательности целых чисел, задающие состояние входа, памяти и выхода.

Команда задаёт отображение на множество состояний: $(n; x; r; y) \rightarrow (n'; x'; r'; y')$.

Команды-операции: (для них всегда $n' = n + 1$)

1) store u — записывает $r(u)$ в ячейку с номером $v(u)$, то есть

$$r'(v(u)) = r(u)$$

2) load u — записывает в нулевую ячейку содержимое ячейки с номером $v(u)$, то есть

$$r'(0) = r(v(u))$$

3) add u — сложение $r'(0) = r(0) + r(u)$

4) sub u — вычитание $r'(0) = r(0) - r(u)$

5) mul u — умножение $r'(0) = r(0) \cdot r(u)$

6) div u — целочисленное деление $r'(0) = r(0) / r(u)$

7) read u — чтение $r'(0) = x(v(u))$

8) write u — запись $y'(v(u)) = r(0)$

Команды-переходы:

- 1) goto u - переход к команде с номером $v(u)$, то есть $n' = v(u)$
- 2) ifzero u - если $r(0) = 0$, то $n' = v(u)$, иначе $n' = n+1$
- 3) ifpos u - если $r(0) > 0$, то $n' = v(u)$, иначе $n' = n+1$

РАМ - это конечная последовательность команд описанного выше вида.

УТВ Множество РАМ счётно.

Машины Микского:

Конечное число регистров $R_1, R_2, \dots, R_n$, каждый из которых содержит целое неотрицательное число.

Обозначим $r(i)$ содержимое регистра с номером i .

Три типа команд:

- 1) inc i, j - увеличить значение регистра i на 1 и передать управление команде с номером j .
- 2) dec i, j, k - если значение регистра i равно 0, то передать управление команде с номером j , иначе уменьшить на 1 значение регистра i и передать управление команде с номером k .
- 3) halt - остановиться

В начальной конфигурации значение всех регистров, кроме первого, равно 0.

Результат работы - значение первого регистра после остановки.

Пример Обнуление регистра

1) $\text{dec } i, 2, 1$

2) halt

Пример: Копирование регистра:

1) $\text{dec } i, 4, 2$

2) $\text{inc } j, 3$

3) $\text{inc } k, 1$

4) halt

Пример: Проверка чётности

1) $\text{dec } i, n, 2$

2) $\text{dec } i, m, 1$

} - уменьшает значение регистра i до 0 и передаёт управление команде с номером n , если в начале работы регистр i содержал чётное число, m - если нечётное.

Пусть A - конечный алфавит.

Опр Нормальным алгоритмом (алгоритмом) Маркова в алфавите A называется конечная последовательность правил подстановки, имеющих вид $u \rightarrow v$ или

$u \rightarrow \epsilon$, где слова $u, v \in A^*$. Правила вида $u \rightarrow \epsilon$ называются ^{закрывающими}

Опр Нормальным алгоритмом над алфавитом A называется такой ^{нормальный} алгоритм

в некотором алфавите B таком, что $A \subseteq B$

Опр Последовательность правил подстановки называется схемой алгоритма.

Применение правила подстановки заключается в замене самого левого вхождения под слова и на под слово v . Если такого под слова нет, то говорят, что рассматриваемое слово не поддается данному правилу.

На каждом шаге применяется первое из правил схемы, применимое к данному слову.

Алгоритм заканчивает работу, когда слово не поддается схеме или после применения заключительного правила.

Лекция 13

Регулярные языки и выражения. ОКА. ИКА. Диаграмма Нура

Опр $\Sigma = \{a_1, a_2, \dots, a_n\}$; $n \in \mathbb{N} \rightarrow$ алфавит, конечное множество

Опр Слово - конечная последовательность символов алфавита

Опр $w[i]$ - i -й элемент слова w

Опр $|w|_a$ - количество букв a в слове w

Опр ε - пустое слово (слово нулевой длины)

Опр w^n - слово, получаемое, если записать слово w ровно n раз подряд

Опр w^R - слово, получаемое, если записать слово w по буквенно в обратном порядке.

Опр Σ^* - множество всех конечных слов над алфавитом Σ .

Опр Языком L над алфавитом Σ называют подмножество Σ^* .

Операции над языками:

1) Конкатенация: $XY = \{xy \mid x \in X, y \in Y\}$

2) Возведение в степень: $X^n = \underbrace{X \cdot X \cdot \dots \cdot X}_n$

3) Объединение: $X \mid Y = X + Y = X \cup Y$

4) Итерация (звезда Клини): $X^* = \varepsilon + X + X^2 + \dots + X^n + \dots$

Пусть задан алфавит Σ . Определим на нём индуктивно регулярные языки и регулярные выражения их задающие.

Регулярный язык

1) $\emptyset \in \text{REG}$

2) $\forall a \in \Sigma, \{a\} \in \text{REG}$

3) $\forall X, Y \in \text{REG} \hookrightarrow XY \in \text{REG}$

4) $\forall X, Y \in \text{REG} \hookrightarrow X+Y \in \text{REG}$

5) $\forall X \in \text{REG} \hookrightarrow X^* \in \text{REG}$

Регулярное выражение

$\emptyset$

a

$X \cdot Y$ или XY

$X+Y$

X^*

Утв Не все языки регулярные

□ Пусть $|\Sigma| = n$. Посчитаем количество регулярных языков. Заметим, что любой регулярный язык определяется формулой, а формул счётно $\Rightarrow |\text{REG}| = \aleph_0$

Теперь посчитаем количество языков. Любое слово можно представить, как

бесконечное число элементов, на каждом из которых написано ϵ или символ из

Σ , т.е. для каждого из $\aleph_0$ положений есть $n+1$ вариант $\Rightarrow (n+1)^{\aleph_0} = \aleph_1 \Rightarrow$

$\Rightarrow$ всего языков несчётно. Ещё можно сказать, что $|\Sigma^*| = \aleph_0$, при этом для

каждого слова есть 2 варианта: включено оно в язык или нет. Получаем то

же самое. Т.к. $\aleph_0 < \aleph_1$, то не все языки регулярные ■

Опр Детерминированный конечный автомат (ДКА) — это набор $(\Sigma; Q; Q_0; \delta; q_f)$,

где:

• Σ — алфавит, слова над которым обрабатывает ДКА

- Q - конечное множество состояний автомата
- Q_f - множество принимающих состояний $Q_f \subseteq Q$
- δ - функция (таблица) переходов $\delta: \Sigma \times Q \rightarrow Q$
- q_0 - начальное состояние. $q_0 \in Q$

Принцип работы:

Читает слово один раз слева направо. Если слово догитано до конца, и ДКА находится в принимающем состоянии, то слово принято, иначе - отвергнуто.

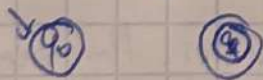
Как выглядит:

ДКА удобно представить как граф (диаграмма Мура):

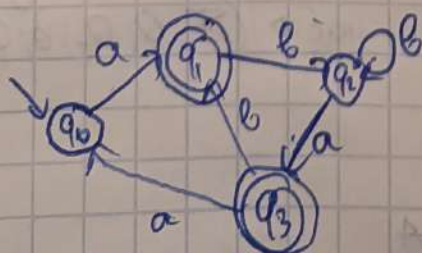
- Каждому состоянию автомата соответствует вершина графа.
- Вершины соединены направленными дугами, помеченными символами алфавита. Они соответствуют функции δ .

$$\delta(a, q_1) = q_2 \Leftrightarrow q_1 \xrightarrow{a} q_2$$

- Начальное состояние обозначают "входящей стрелкой", а принимающее - двойной обводкой.



Пример ДКА:



Полный ДКА-ДКА, в котором из каждой вершины выходят рёбра со всеми символами алфавита.

Чтобы его построить по ДКА введём дополнительную вершину $\rightarrow \textcircled{T}$ - помойка (Brawl Stars). В неё проведём все отсутствующие рёбра, и петли по всем символам. Из $\textcircled{T}$ нельзя выбраться, и она отвергает слово

Опр Недетерминизированный конечный автомат (НКА) - это набор $(\Sigma, Q, Q_f, \delta, q_0)$

где:

- Σ - алфавит
- Q - конечное множество состояний автомата
- Q_f - множество принимающих состояний: $Q_f \subseteq Q$
- δ - функция переходов $\delta: \{\Sigma \cup \epsilon\} \times Q \rightarrow 2^Q$
- q_0 - начальное состояние, $q_0 \in Q$

Функция переходов у НКА неоднозначная.

НКА принимает слово, если возможно достижение принимающего состояния при полном считывании слова.

Факт Недетерминизированные конечные автоматы и машины Тьюринга порождают мультиязыки.

Лемма Проверка принятия слова НКА конечна. (Ситуация некатастрофическая)

□ Пусть $|\Sigma| = n$. Тогда между $\forall$ двумя буквами $\leq n$ ϵ -переходов $\Rightarrow$ перебор конечный

Лекция 14

Эквивалентность РВ, ДКА и НКА. Лемма о накачке. Теорема

Майхилла - Нероуд. Замкнутость регулярных языков

Утв Регулярные выражения, детерминированные конечные автоматы и недетерминированные конечные автоматы задают одни и те же сущности.

□ Доказательство будет конструктивным. Будем строить по РВ $\rightarrow$ НКА;

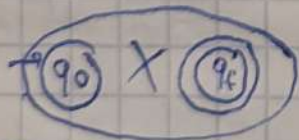
НКА $\rightarrow$ ДКА; ДКА $\rightarrow$ РВ ■

Построение НКА по РВ

РВ строятся индуктивно. Будем строить НКА по РВ в том же порядке, соблюдая следующие ограничения:

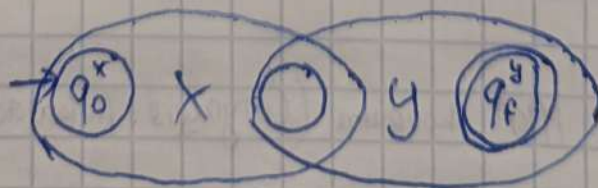
- 1) НКА имеет ровно одно принимающее состояние
- 2) В начальное состояние НКА не ведёт ни один переход
- 3) Из принимающего состояния НКА нет ни одного перехода

Будем изображать НКА языка X схематично так:

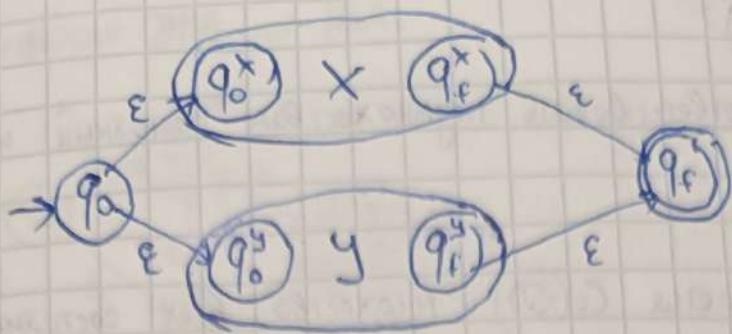


Построим НКА для операций:

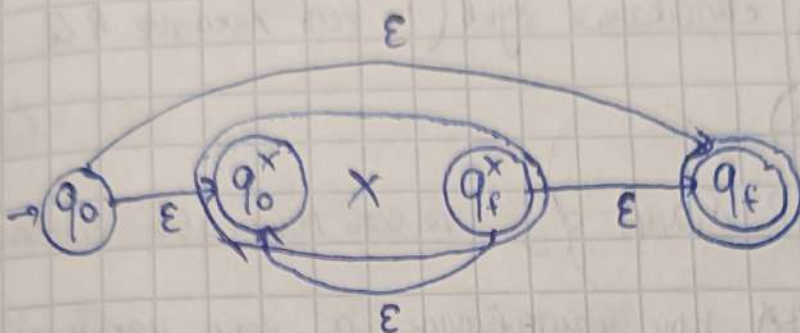
1) XY :



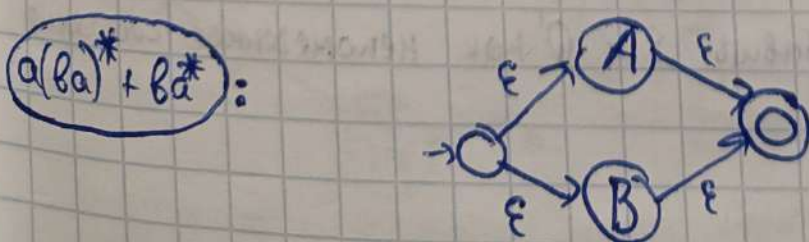
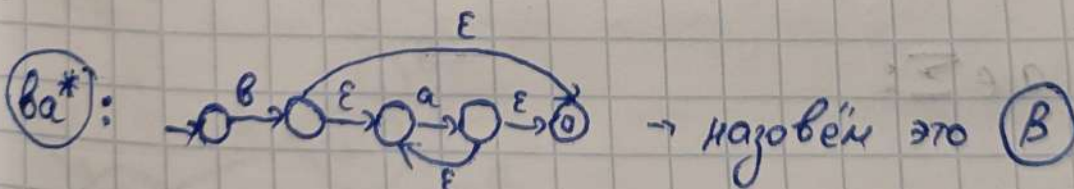
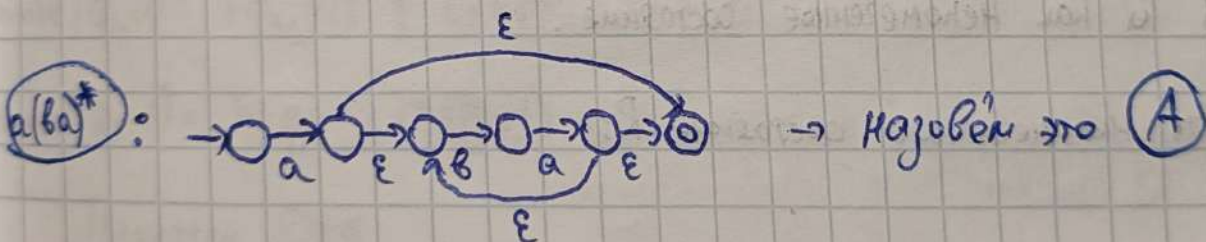
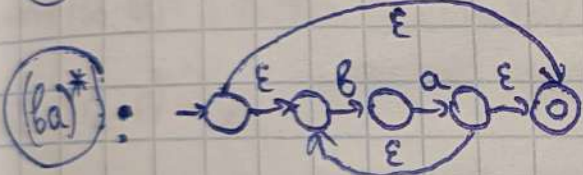
2) $x+y$:



3) x^* :



Пример. Построим НКА по $a(ba)^* + ba^*$



Построение ДКА по НКА:

Состояния ДКА будут соответствовать подмножествам состояний исходного НКА.

Опр ϵ -замыкание (обозначается $\epsilon(Q)$) - множество всех состояний, достижимых из всех $q \in Q$ без считывания букв (то есть лежащие в Q или достижимые по ϵ -переходам)

Опр $\text{move}(R, a) = \{q' \mid \exists q \in R: \delta(a, q) = q'\}$ - то есть множество состояний, в которые может перейти НКА при считывании a , если находился в одном из состояний $R \subseteq Q$

Алгоритм построения ДКА по НКА:

Сначала Q' и δ' пусты.

1) Определить $q'_0 = \epsilon(q_0)$

2) Добавить q'_0 в Q' как непомеченное состояние.

3) Пока в Q' есть непомеченное состояние R :

- Пометить R

- Для каждого символа $a \in \Sigma$:

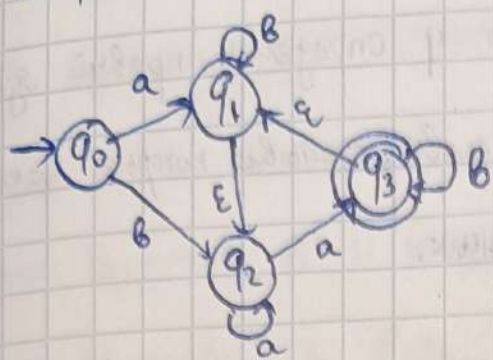
- $S = \epsilon(\text{move}(R, a))$

- Если $S \neq \emptyset$ и $S \notin Q'$, то добавить S в Q' как непомеченное состояние и определить $\delta'(a, R) = S$

4) Определить $Q'_f = \{S \mid S \in Q', S \cap Q_f \neq \emptyset\}$

Пример построения ДКА по НКА:

НКА:



$Q_0 = C_\epsilon(q_0) = \{q_0\}$

$move(Q_0, a) = \{q_1\} \quad C_\epsilon(\{q_1\}) = \{q_1, q_2\} = Q_1$

$move(Q_0, b) = \{q_2\} \quad C_\epsilon(\{q_2\}) = \{q_2\} = Q_2$

$move(Q_1, a) = \{q_1, q_2\} \quad C_\epsilon(\{q_1, q_2\}) = \{q_1, q_2, q_3\} = Q_3$

$move(Q_1, b) = \{q_1\} \quad C_\epsilon(\{q_1\}) = \{q_1, q_2\} = Q_1$

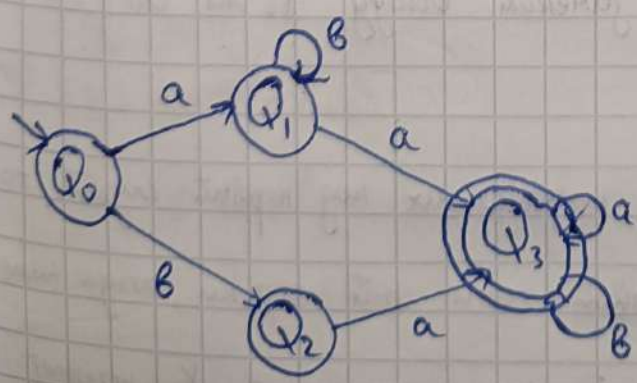
$move(Q_2, a) = \{q_2, q_3\} \quad C_\epsilon(\{q_2, q_3\}) = \{q_1, q_2, q_3\} = Q_3$

$move(Q_2, b) = \emptyset$

$move(Q_3, a) = \{q_2, q_3\} \quad C_\epsilon(\{q_2, q_3\}) = Q_3$

$move(Q_3, b) = \{q_1, q_3\} \quad C_\epsilon(\{q_1, q_3\}) = \{q_1, q_2, q_3\} = Q_3$

Получаем ДКА:



Построение РВ по ОКА

Опр По заданному НКА/ОКА для каждого q определим правый язык $R_q = \{x \mid q \xrightarrow{x} q_f; q_f \in Q_f\}$, то есть все слова, считывая которые можно попасть из состояния q в одно из финальных.

Опр Назовём

$$\begin{cases} X_1 = a_{11}X_1 + a_{12}X_2 + \dots + a_{1n}X_n + \beta_1 \\ \vdots \\ X_n = a_{n1}X_1 + a_{n2}X_2 + \dots + a_{nn}X_n + \beta_n \end{cases}$$

системой уравнений с регулярными коэффициентами, где X_i - переменные-множества, $a_{ij}; \beta_i$ - регулярные языки, задаваемые регулярными выражениями

Опр Решением системы с языками-переменными $X_1; X_2; \dots; X_n$ будем называть такой набор языков $A_1; A_2; \dots; A_n$, удовлетворяющий системе, что для $\forall$ набора языков $B_1; B_2; \dots; B_n$, удовлетворяющего системе, выполнено $A_i \subseteq B_i \quad \forall i \in \{1, \dots, n\}$

Алгоритм решения:

Случай 1: Если в системе есть строка вида:

$X_i = a_{ii}X_i + f$, где $a_{ii} \neq \emptyset$, то заменим всюду X_i на a_{ii}^*f , а

саму строку удалим

Случай 2: Если в системе нет строк, подходящих под первый случай, то переменная X_n не встречается в правой части n -й строки, тогда надо

заменить всюду X_n на правую часть n -ой строки. переменная X_n исчезнет

Алгоритм построения РВ по ОКА:

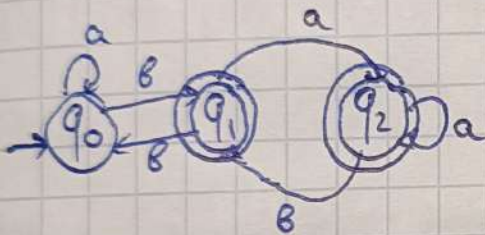
В качестве переменных при составлении системы будем использовать правые языки всех состояний.

Для каждого состояния q_i и каждого символа a ; если из состояния q_i есть переход по символу a в q_j , то к правой части строки R_{q_i} добавим aR_{q_j} , к правым частям финальных состояний добавим ϵ .

Решим систему ~~из уравнений~~ ~~из равенств~~ подставим все переменные. Ответом будет R_0 .

Пример построения РВ по ОКА:

ОКА:



Составим систему:

$$\begin{cases} R_0 = aR_0 + bR_1 \\ R_1 = aR_2 + bR_0 + \epsilon \\ R_2 = aR_2 + bR_1 + \epsilon \end{cases} \Leftrightarrow \begin{cases} R_0 = a^*bR_1 \\ R_1 = ba^*bR_1 + aR_2 + \epsilon \\ R_2 = bR_1 + aR_2 + \epsilon \end{cases} \Leftrightarrow \begin{cases} R_0 = a^*bR_1 \\ R_1 = (ba^*b)^*(aR_2 + \epsilon) \\ R_2 = b(ba^*b)^*(aR_2 + \epsilon) + aR_2 + \epsilon \end{cases} \Leftrightarrow$$

$$\Leftrightarrow \begin{cases} R_0 = a^*bR_1 \\ R_1 = (ba^*b)^*(aR_2 + \epsilon) \\ R_2 = (b(ba^*b)^*a + a)R_2 + b(ba^*b)^* + \epsilon \end{cases}$$

Подставим R_2 и потом R_1 и найдём R_0 .

Теорема (лемма о накачке/разрастании)

Пусть $L \subseteq \Sigma^*$ - регулярен. Тогда $\exists$ такое $k \in \mathbb{N}$: $\forall w \in L$: $|w| > k \Rightarrow$ слова

x, y, z : $w = xyz$ и:

- 1) $|y| > 0$
- 2) $|xy| \leq k$
- 3) $xy^i z \in L \quad \forall i \in \mathbb{N} \cup \{0\}$

Зам Это не критерий. Пример нерегулярного языка, удовлетворяющего условиям

леммы:

$$L = \{a^i b^j c^k \mid i \neq 1; j \geq 0; k \geq 0\} \cup \{a b^i c^i \mid i \geq 1\}$$

Теорема (Теорема Майхилла-Нерода)

Пусть дан $L \subseteq \Sigma^*$. Определим отношение Майхилла-Нерода ($\equiv_L$) на Σ^*

следующим образом: $x \equiv_L y \Leftrightarrow \forall z \in \Sigma^* : xz \in L \Leftrightarrow yz \in L$

Тогда:

Язык L регулярен $\Leftrightarrow$ число классов эквивалентности по отношению Майхилла-Нерода

конечно.

$\square \Rightarrow$ Язык L - регулярен $\Rightarrow \exists$ ПКА, соответствующий этому языку. Тогда слова x и y эквивалентны, если при запуске $\min$ ПКА на словах x и y он останавливается в одном и том же состоянии. Т.к. состояний конечно, то и классов эквивалентности конечно

$\Leftarrow$ Т.к. конечное число классов, то можно для каждого класса завести вершину

в ПКА. Для слова x из каждого класса проведём рёбра со всеми символами в те вершины, в которых лежат xa , где a тот символ. Тогда можно отметить тот класс, в котором есть ϵ за начальное состояние. А за финальные - те, в которых есть слова из L ■

Замкнутость регулярных языков:

Пусть $A, B \in REG$ над Σ^* . Тогда:

- 1) $A+B, AB, A^* \in REG$
- 2) $\bar{A} = \Sigma^* \setminus A \in REG$
- 3) $A \cap B \in REG$
- 4) $A \setminus B \in REG$
- 5) Можно получить регулярный язык, сделав гомоморфизм и реверс.

Лекция 15

Грамматика

Опр Грамматика - это четверка (N, T, P, S) , где

N - алфавит нетерминальных символов

T - алфавит терминальных символов

P - конечное множество правил вида $\alpha \rightarrow \beta$, где $\alpha \in (NUT)^* N (NUT)^*$; $\beta \in (NUT)^*$

$S \in N$ - начальный знак (аксиома грамматики)

Типы грамматик:

Тип 0: Нет дополнительных ограничений (практического применения не имеют, т.к. слишком сложная структура, распознаётся МТ.)

Тип 1 (контекстно-зависимая, неукорачивающая): каждое правило грамматики кроме $S \rightarrow \epsilon$ имеет вид $\alpha \rightarrow \beta$, где $|\alpha| \leq |\beta|$, и в случае, если правило $S \rightarrow \epsilon \in P$, то символ S не встречается в правых частях правил (используются для анализа естественных языков)

Тип 2 (контекстно-свободная): каждое правило имеет вид $A \rightarrow \beta$, где $A \in N$; $\beta \in (NUT)^*$ (применяются для описания синтаксиса компьютерных языков, распознаются при помощи конечного автомата со стеком).

Тип 3 (регулярная): каждое правило грамматики имеет вид $A \rightarrow x\beta$ или $A \rightarrow x$, где $A, \beta \in N$; $x \in T^*$ (совпадают со множеством регулярных. Используются для ассемблера, командных процессов).